

VOXGEST

Deep Comprehensive Contextual Notes

Technical system overview, approach decisions, model logic, software tools, architecture relationships, and Chapter 1-3 paper alignment

Document purpose

This document is different from the simplified contextual notes. The simplified note explains the project like a beginner story. This comprehensive note explains the same project as an IT Specialist or Senior Software Engineer would review it, but still uses simple anchors and analogies so every teammate can understand the logic.

Main rule

The system story must be honest: VoxGest is a controlled, offline Android accessibility prototype. It recognizes static alphabet/control signs, selected dynamic words, and no-output movement. It does not claim full ASL grammar, unlimited vocabulary, or production-level interpreter replacement.

Manual Table of Contents

1. Executive System Summary
2. Chapter 1-3 Logic: How the Paper and System Must Match
3. Full Architecture Map
4. Tools, Software, Libraries, and Runtime Roles
5. Data Strategy and Dataset Logic
6. Model Evolution and Why Each Approach Changed
7. Deep Decision Cards: Used, Rejected, Delayed, or Experimental
8. Recognition Hardening: NOTHING, Gates, and Token Composer
9. Android/TFLite Runtime Contract
10. Avatar, Speech, and Bidirectional Communication
11. Phrase Intents and Why They Are Frozen
12. FullSign225 and Residual Dilated TCN Experiments
13. Paper Holes and How to Fix/Explain Them
14. Defense-Ready Technical Answers
15. Current Limitations and Next Engineering Path
16. Final Logical Summary
17. Technical Pipeline Traces
18. Preprocessing: The Hidden Part That Can Break the Whole System
19. Training, Validation, and Live Testing Relationship
20. Relevance and Irrelevance of Alternative Approaches
21. Chapter 1 Detailed Alignment Notes
22. Chapter 2 Detailed Alignment Notes
23. Chapter 3 Detailed Alignment Notes
24. Engineering Risk Register
25. Glossary for Technical Review

Executive System Summary

VoxGest is an offline Android edge-computing accessibility prototype. Its purpose is to help a sign language user and a hearing or non-signing person communicate through one mobile device. The camera reads controlled signs. The system turns only accepted signs into text and speech. The hearing person can speak back, and the app can show text, lightweight avatar feedback, or fingerspelling.

This document explains the technical logic behind Chapter 1 to Chapter 3. It is written for a technical reviewer, but it still uses simple explanations so the team can defend the decisions orally without sounding memorized or confused.

Donkey-level system picture

Imagine the phone is a small translator booth. The camera is the eye. MediaPipe is the dot maker. The model is the trained guesser. Gates are the security guard. The token composer is the sentence builder. TTS is the loudspeaker. STT is the ear. The avatar is the visual reply helper.

Current Honest State

- **Static recognition:** A-Z, del, space, and nothing are treated as the spelling/control foundation.
- **Dynamic recognition:** The protected baseline is the controlled demo10 word set: YES, NO, PLEASE, WATER, HELLO, HELP, STOP, DOCTOR, NAME, THANKYOU, plus NOTHING.
- **Output safety:** Only accepted tokens should update the sentence strip, trigger speech, or trigger avatar playback.
- **Android direction:** Training happens in Python. Android consumes exported TFLite models, labels, and runtime contracts.
- **Current engineering priority:** Recognition Hardening v1. This means stabilize alphabet, current words, NOTHING, gates, token composer, and avatar contract before expanding vocabulary or phrase recognition.

What This Note Is Not

- It is not a replacement for the simplified contextual notes. The simplified note stays as is.
- It is not a marketing document. It is an engineering and paper-alignment document.
- It is not claiming the system is already production-ready.
- It is not claiming full ASL translation or unlimited sentence understanding.

Chapter 1-3 Logic: How the Paper and System Must Match

The paper must not describe one system while the code and demo show another. The main defense risk is not only technical accuracy; it is story consistency. Chapter 1 says what problem is being solved, Chapter 2 justifies the technologies, and Chapter 3 explains how the system is built and evaluated. If those chapters do not align, the panel can easily find contradictions.

Chapter	What it must prove	How VoxGest should connect
Chapter 1	The communication problem is real and the proposed system has a clear, limited, useful scope.	Present VoxGest as a bidirectional offline Android accessibility prototype, not a full ASL translator. Include static signs, selected dynamic words, NOTHING, token composer, speech, and avatar feedback.
Chapter 2	The related literature supports the actual technologies and design choices.	Discuss sign language recognition, MediaPipe landmarks, sequence models, TCN/LSTM, edge computing, Android/TFLite, negative/no-output classes, and software evaluation.
Chapter 3	The methodology explains the build process, datasets, feature engineering, model evolution, testing, Android handoff, and evaluation.	Use developmental research and Agile RAD because the system was improved through repeated model-building, live testing, and revision cycles.

The Paper Story in One Sentence

VoxGest was developed as a controlled, offline Android assistive communication prototype that uses camera-based landmarks, static and temporal recognition models, output-safety gates, token composition, speech output, speech-to-text, and lightweight avatar/fingerspelling feedback to support basic two-way communication.

Why this sentence matters

This sentence connects the problem, the technology, the methodology, and the limitation. It avoids the old mistake of calling VoxGest only a CNN alphabet recognizer.

Full Architecture Map

The system is best understood as layers. Each layer solves a specific problem and passes a cleaner result to the next layer. A raw camera frame should never jump directly to spoken output. It must pass through landmarks, model inference, gates, stability checks, token composition, and only then user-facing output.

Recognition flow:

- Camera frame
- > MediaPipe landmarks
- > feature extractor
- > static or dynamic model
- > confidence/motion/quality gates
- > stable accepted token
- > token composer
- > text / speech / avatar output

Speech response flow:

- Hearing person speaks
- > speech-to-text
- > displayed text
- > known word animation or fingerspelling
- > visual feedback to signing user

Layer-by-Layer Explanation

Layer	Technical role	Simple meaning
Camera	Captures live video frames from Android CameraX or Python webcam.	The eye of the system.
Landmarks	MediaPipe extracts hand/pose coordinates from image frames.	The dot maker. It turns pictures into points.
Features	Coordinates are normalized and shaped into model inputs.	The translator between dots and AI numbers.
Models	Static classifier, LSTM/TCN, and experimental FullSign225/RD-TCN paths classify controlled signs.	The trained guessers.
Gates	Confidence, margin, motion, wrist path, hand presence, quality, stable frames, and cooldown reduce false outputs.	The security guard.
Token composer	Builds sentence text only from accepted labels.	The sentence builder.
Speech/avatar	TTS speaks confirmed signed text; STT captures spoken reply; avatar/fingerspelling gives visual feedback.	The mouth, ear, and visual helper.

Tools, Software, Libraries, and Runtime Roles

The system uses separate tools for research development, model training, live testing, Android deployment, and documentation. This separation is important. It prevents the team from assuming that because something works in Python, it automatically works in Android.

Tool / Software	Where used	Role in VoxGest
Python	ML development PC	Dataset preparation, feature extraction, model training, diagnostics, live prototype testing.
VSCode	Development environment	Code editing and project organization.
Miniconda / virtual environment	Python environment management	Stabilizes TensorFlow, MediaPipe, NumPy, OpenCV, and protobuf compatibility.
OpenCV	Python camera/video processing	Reads webcam frames and video files, handles frame operations.
MediaPipe Hands / Holistic	Feature extraction	Extracts hand, pose, and body landmarks used by models.
TensorFlow / Keras	Training environment	Builds and trains static/dynamic models.
TensorFlow Lite	Android runtime target	Runs exported models locally on mobile device.
Kotlin	Android application	Main app language for UI and inference integration.
CameraX	Android camera layer	Provides camera preview and frame analysis pipeline.
Android SpeechRecognizer / TTS	Bidirectional communication	Speech-to-text and text-to-speech modules.
JSON manifests	Runtime contract	Stores labels, model shapes, thresholds, and avatar dictionary rules.
Overleaf / LaTeX template	Research paper	Maintains institutional formatting while updating Chapter 1-3 content.

Why separate Python and Android?

Python is the kitchen where models are prepared and tested. Android is the restaurant where the user receives the finished meal. Training models on Android is not the current goal because it increases complexity and device load. Android should only run exported models.

Data Strategy and Dataset Logic

The quality of a sign recognition system depends heavily on the data. VoxGest has used several data strategies over time because each stage had a different recognition problem.

Static alphabet data

The static alphabet path started from image datasets for A-Z plus control signs such as del, space, and nothing. This made sense for early CNN training because static image datasets are easier to collect and classify. However, once live testing exposed image-domain problems, the static path shifted to landmark features.

Dynamic word data

Dynamic signs are not just hand shapes. They are short movements. A dynamic word sample must preserve motion over time, so VoxGest converts signs into fixed-length sequences. The protected demo10 path uses 30 frames per word decision. The experimental phrase path uses 60 frames because phrases are longer and need endpoint segmentation.

Manual recording and team recorder pack

Manual recording became important because external datasets may not match the live webcam or Android camera style. A model trained only on public videos may perform well on saved data but fail during live demonstration. The team recorder pack allows teammates to capture landmark samples, while one main PC merges, audits, trains, and exports the final models. This prevents each teammate from accidentally creating incompatible models.

Why clean metadata matters

Metadata such as dominant hand, mirrored input, and single-hand pose policy tells the training pipeline how to interpret a sample. Without metadata, the same physical hand can be interpreted differently across webcam and video sources. This caused earlier one-hand inconsistencies and is why ambiguous samples should be archived or repaired.

Data quality rules

- Do not mix left-hand and right-hand samples in the same controlled recording command.
- Do not use auto-hand mode for controlled recording because it can switch hands during capture.
- Record NOTHING as idle, transition, partial, aborted, and non-word movement, not as a word.
- Collect multiple recording groups, not just one long group, because validation should test generalization across different sessions.
- Audit dataset counts before training to avoid missing classes or accidental output-shape changes.

Model Evolution and Why Each Approach Changed

The model evolution is not a weakness. It is evidence that the team tested, found limitations, and improved the architecture. In an oral defense, this should be explained as engineering learning and system hardening.

Stage	Approach	Why it was logical	Problem found	Decision
1	Image CNN	CNNs are standard for image classification and static hand-shape learning.	Webcam background, lighting, cropping, and domain gap affected live recognition.	Keep as early research path, not final system core.
2	Hybrid CNN + landmarks	Tried combining visual image features with hand geometry.	More complexity did not solve live fragility and introduced data leakage/memorization risk.	Abandoned for production direction.
3	Landmark-only static classifier	Hand geometry is more stable than raw pixels for static signs.	Requires exact preprocessing match between Python and Android.	Use as static alphabet/control foundation.
4	Hand-only dynamic features	Simple extension from hand landmarks to word signs.	Floating-hand problem: model did not know if hand was near face, chin, or chest.	Expand to pose + hand features.
5	Onehand162 LSTM/TCN	Adds body-relative location and motion over time while preserving one-hand accessibility.	Some words still confuse; live behavior differs from validation.	Harden with NOTHING, gates, targeted repair data, and TCN comparison.
6	Phrase-intent path	Longer sentence shortcuts need complete gesture segmentation.	Premature recognition when tested like short words.	Freeze during Recognition Hardening v1; keep for future milestone.
7	FullSign225 + RD-TCN experiment	Some full signs benefit from pose + both hands and stronger temporal model.	Requires more clean data and live comparison before default use.	Keep separate from protected demo10 baseline.

Deep Decision Cards: Used, Rejected, Delayed, or Experimental

This section explains every major engineering decision. Each card answers: What approach we used, why we used it, why not the alternative, how it works, and how it connects to Chapter 1 to Chapter 3.

Decision 1: Use a phone-based accessibility app

Plain-English anchor

One phone acts like a small bridge between a signer and a hearing person.

Approach used	Android/mobile application as the main user platform.
Why this was used	Phones are common, portable, and appropriate for daily public settings like schools, hospitals, offices, and counters.
Why not the other approach right now	A desktop-only system would be easier to prototype but less useful in real daily communication. Gloves or special sensors would add cost and reduce accessibility.
How it works technically	The Android device captures camera frames, runs local inference, shows text, speaks output, and displays visual feedback.
Logical relationship to the system	Every technical module exists to support the user-facing communication bridge.
Chapter 1-3 connection	Chapter 1 explains the problem and need; Chapter 3 explains the Android system architecture.

Decision 2: Use offline/on-device recognition

Plain-English anchor

The phone should think locally instead of asking the internet every time.

Approach used	TensorFlow Lite on Android with no backend required for recognition.
Why this was used	Offline use supports privacy, lower latency, and operation in places with weak internet.
Why not the other approach right now	Cloud APIs may be powerful but introduce internet dependency, privacy concerns, cost, and latency. They also weaken the local-only capstone objective.
How it works technically	Python exports .tflite models and labels. Android loads them using a TFLite interpreter.
Logical relationship to the system	This connects accessibility with edge computing: fast, private, local, and portable.
Chapter 1-3 connection	Chapter 2 supports edge deployment; Chapter 3 describes the Android/TFLite pipeline.

Decision 3: Start with CNN, then move beyond CNN-only

Plain-English anchor

The first AI looked at hand pictures, but live camera reality was messier.

Approach used	CNN was used as an initial image-based recognizer, then reduced to a historical/literature role.
Why this was used	CNNs are standard for static image classification and are academically understandable for early ASL alphabet recognition.
Why not the other approach right now	CNN-only is not reliable enough for this live prototype because background, lighting, camera angle, cropping, and hand distance can distort raw pixels.
How it works technically	The project used the CNN attempt to learn the limits of raw image recognition, then shifted to landmarks for live robustness.
Logical relationship to the system	This decision shows iterative development based on evidence, not random changes.

Chapter 1-3 connection	Chapter 2 discusses CNNs as related work; Chapter 3 explains model evolution.
-------------------------------	---

Decision 4: Use MediaPipe landmarks as the main feature strategy

Plain-English anchor

Instead of staring at the whole room, the system follows important dots.

Approach used	MediaPipe hand and pose landmarks are used as numerical features.
Why this was used	Landmarks reduce the impact of background and lighting because the model receives coordinates instead of full images.
Why not the other approach right now	Raw images force the model to learn irrelevant visual noise. Manual segmentation would be time-consuming and fragile.
How it works technically	MediaPipe extracts x, y, z landmark coordinates. These are normalized and shaped into model input arrays.
Logical relationship to the system	Landmarks are the bridge between camera frames and machine learning models.
Chapter 1-3 connection	Chapter 3 feature engineering depends on this approach.

Decision 5: Use 63 features for static alphabet/control signs

Plain-English anchor

One hand has 21 dots; each dot has x, y, z; $21 \times 3 = 63$.

Approach used	Static model uses normalized 63 hand-landmark features.
Why this was used	Static signs depend mostly on hand shape, so a compact hand geometry representation is sufficient and efficient.
Why not the other approach right now	Using full body features for every static letter would add unnecessary noise and computational cost. Using raw images would reintroduce lighting/background problems.
How it works technically	Subtract wrist position and scale by hand size so the same sign stays similar at different distances.
Logical relationship to the system	This creates the spelling/control foundation of the app.
Chapter 1-3 connection	Chapter 1 scope and Chapter 3 static recognition method must mention 63-feature static landmarks.

Decision 6: Use 30 x 162 for dynamic words

Plain-English anchor

A moving sign is like a short movie, not a single photo.

Approach used	Dynamic word model uses 30 frames, each with 99 pose + 63 dominant-hand features.
Why this was used	Dynamic signs require time and body-relative location. Pose shows where the hand is near the face/chest/body; hand landmarks show the motion.
Why not the other approach right now	A single static frame cannot capture motion. Hand-only 63 features caused the floating-hand problem. Full two-hand 225 features are promising but still experimental for current baseline.
How it works technically	Each word attempt becomes a 30-frame sequence. Each frame has 162 features.
Logical relationship to the system	This gives the model motion and body context while keeping the protected one-hand accessibility direction.
Chapter 1-3 connection	Chapter 3 dynamic feature engineering and Android contract depend on this exact shape.

Decision 7: Use LSTM and TCN for motion, with TCN as current main direction

Plain-English anchor

The model must watch movement in order.

Approach used	LSTM and TCN are sequence models; TCN is the current main dynamic training direction while LSTM remains comparison/back-up.
Why this was used	They can learn motion patterns from 30-frame landmark sequences. TCN is lightweight and suitable for fixed-length sequences and TFLite deployment.
Why not the other approach right now	A static dense model cannot understand movement order. A large Transformer/ST-GCN could be powerful but needs more data, compute, and time than current scope allows.
How it works technically	Training scripts read sequence arrays, train temporal models, export .h5/.tflite, and compare reports and live behavior.
Logical relationship to the system	This connects dynamic recognition with practical mobile deployment.
Chapter 1-3 connection	Chapter 2 should justify sequence models; Chapter 3 should describe TCN/LSTM model development.

Decision 8: Add NOTHING as a negative/no-output class

Plain-English anchor

The system needs to learn when to shut up.

Approach used	NOTHING is trained/handled as no valid word, not as a spoken word.
Why this was used	Softmax classifiers tend to choose a class even from idle movement. NOTHING teaches the model that idle, partial, aborted, or transition movement should produce no output.
Why not the other approach right now	Without NOTHING, gates alone may reject some weak motions but the model still lacks examples of non-word behavior. Calling it a normal word would be wrong because it should never be spoken or animated.
How it works technically	Record idle hands, open hand, transitions, partial signs, aborted signs, hand entering/leaving frame. Treat output as no-op.
Logical relationship to the system	This is a safety layer against embarrassing or harmful false outputs.
Chapter 1-3 connection	Chapter 1 scope, Chapter 2 negative-class discussion, and Chapter 3 testing must include NOTHING.

Decision 9: Use gates before token output

Plain-English anchor

The AI guess is not automatically trusted.

Approach used	Confidence, margin, motion, wrist path, hand presence, quality, stability, and cooldown are checked before output.
Why this was used	A prediction can be mathematically high but still unsafe if the hand is missing, motion is too weak, or the model is unstable across frames.
Why not the other approach right now	Outputting every raw prediction would create random text, speech, and avatar triggers. Depending only on validation accuracy ignores live camera problems.
How it works technically	The runtime computes metrics and accepts a label only after thresholds and stability rules pass.
Logical relationship to the system	Gates separate model prediction from user-facing communication.
Chapter 1-3 connection	Chapter 3 must describe post-processing and testing, not just

model architecture.

Decision 10: Use token composer instead of raw prediction display**Plain-English anchor**

Only approved words enter the sentence.

Approach used	Accepted letters and words build a sentence strip; rejected/raw predictions do not modify output.
Why this was used	This prevents flickering model guesses from becoming spoken or displayed text.
Why not the other approach right now	Directly speaking model predictions would be unsafe and confusing. A chat-like sentence builder gives better control.
How it works technically	A-Z appends characters, space commits a word boundary, del deletes, word labels append full tokens, NOTHING is ignored.
Logical relationship to the system	The composer is the final safety layer before text, TTS, and avatar.
Chapter 1-3 connection	Chapter 1 purpose and Chapter 3 system architecture should mention token-based output.

Decision 11: Prioritize dominant-hand and one-hand policy**Plain-English anchor**

The app focuses on one reliable signing hand first.

Approach used	Dominant-hand selection and single-hand pose masking guide training/testing/runtime.
Why this was used	This supports accessibility for users with one functional hand and keeps the model simpler for real-time mobile use.
Why not the other approach right now	Full two-hand recognition is broader but harder, needs more data, and can conflict with the current one-hand accessibility goal. Auto-hand mode is useful for testing but risky for controlled recording.
How it works technically	Use right/left configured hand during recording; mask non-dominant arm landmarks when single-hand policy is enabled.
Logical relationship to the system	This aligns data collection, training, testing, and Android preprocessing.
Chapter 1-3 connection	Chapter 1 accessibility rationale and Chapter 3 feature policy should include this decision.

Decision 12: Delay phrase intents during hardening**Plain-English anchor**

Do not ask the system to write sentences before it can reliably say words.

Approach used	Phrase-intent architecture exists but is off by default during Recognition Hardening v1.
Why this was used	The first phrase test showed premature recognition because the phrase was treated like a short word. Longer gestures need endpoint segmentation and more data.
Why not the other approach right now	Enabling phrase recognition now can distract from stabilizing static signs, demo10, NOTHING, and token composer.
How it works technically	Phrase path should classify only after IDLE -> START_MOTION -> ACTIVE_MOTION -> END_HOLD and 60-frame segment resampling.
Logical relationship to the system	This shows controlled project management: stabilize foundations before expansion.
Chapter 1-3 connection	Chapter 1 limitations and Chapter 3 future work should mention phrase intents are explored but disabled.

Decision 13: Keep FullSign225 and Residual Dilated TCN experimental

Plain-English anchor

A stronger engine should not replace a safe engine until it proves itself.

Approach used	FullSign225 uses 225 features; RD-TCN is a stronger temporal model experiment.
Why this was used	Some signs may need both hands and wider time context. These experiments may improve future recognition.
Why not the other approach right now	Making them default too early could break the protected baseline, change model contracts, and create paper/demo inconsistency.
How it works technically	Train in separate profiles/scripts/artifacts; compare with live logs; promote only after evidence.
Logical relationship to the system	This protects demo10 while allowing technical growth.
Chapter 1-3 connection	Chapter 3 can mention them as experimental improvement paths, not as final default.

Decision 14: Add lightweight avatar/fingerspelling downstream of accepted tokens

Plain-English anchor

The hearing person can speak back, so the signer needs a visual reply.

Approach used	Known words can play simple avatar animations; unknown words can be fingerspelled.
Why this was used	Bidirectional communication is more complete than sign-to-speech only.
Why not the other approach right now	A full ASL grammar avatar is beyond current scope. Triggering avatar from raw predictions would be unsafe.
How it works technically	Avatar reads confirmed tokens/text after composition. NOTHING triggers no animation.
Logical relationship to the system	This completes the communication loop without overclaiming full sign generation.
Chapter 1-3 connection	Chapter 1 purpose and Chapter 3 avatar/speech module should include it.

Recognition Hardening: NOTHING, Gates, and Token Composer

Recognition Hardening v1 means the team is intentionally prioritizing reliable output over feature expansion. This is the correct engineering move because adding vocabulary before fixing false positives can make the system look bigger but less trustworthy.

Why hardening matters technically

- **Model probability is not truth:** A classifier can output a confident answer even when the user is not signing clearly.
- **Live camera differs from saved validation data:** Lighting, distance, hand visibility, and motion speed can change during real use.
- **User-facing speech is risky:** If the app speaks a wrong word aloud, the communication error is more visible than a wrong debug prediction.
- **Avatar output must be protected:** A wrong animation can mislead the signing user.

The hardening chain

Raw model prediction

-> confidence check

- > margin check
- > motion / wrist path check
- > hand presence / landmark quality check
- > stable frames
- > cooldown
- > accepted token
- > token composer
- > text / TTS / avatar

Why validation accuracy alone is not enough

Validation accuracy measures performance on held-out saved samples. It does not fully measure camera placement, mirroring, hand switching, partial gestures, hesitation, user fatigue, or transition movement. That is why the live logger and diagnostic scripts are important. They reveal why a prediction is accepted or rejected during real testing.

Android/TFLite Runtime Contract

The Android app must copy the Python preprocessing rules. The model does not understand the word YES or WATER directly. It understands numbers shaped exactly like the training data. If Android gives the model different numbers for the same sign, recognition can fail even if the model file is correct.

Core Android responsibilities

- Use front camera and mirror behavior consistently.
- Extract hand and pose landmarks with the same logic as Python.
- Choose the same dominant hand policy.
- Normalize static hand features the same way.
- Build 30-frame dynamic sequences with the same 162-feature order.
- Load the correct TFLite model and matching label file.
- Apply the same gates, smoothing, stable frames, and cooldown rules.
- Send only accepted tokens to the composer, TTS, and avatar.

Runtime item	Static model	Dynamic model
Input shape	[1, 63]	[1, 30, 162]
Output	A-Z, del, space, nothing	YES, NO, PLEASE, WATER, HELLO, HELP, STOP, DOCTOR, NAME, THANKYOU, NOTHING
Feature source	21 hand landmarks	33 pose landmarks + selected/dominant hand landmarks
Main risk	Wrong normalization or low confidence	Wrong hand selection, missing pose, motion confusion, false positives
Output handling	Accepted character/control token	Accepted word token or no-op NOTHING

Donkey-level Android warning

If Python trained the model using apples, Android must also feed apples. If Android feeds oranges because it mirrors the camera differently or orders landmarks differently, the model will look broken even if the file is correct.

Avatar, Speech, and Bidirectional Communication

The original one-way idea was simple: signer signs, app speaks. The problem is that this is not a real conversation. The hearing person also needs a way to reply in a form the signing user can access. That is why the system includes speech-to-text and avatar/fingerspelling feedback.

Communication outputs

- **Text output:** Displays accepted sentence text for direct reading.
- **Text-to-speech:** Speaks confirmed signed output to the hearing/non-signing person.
- **Speech-to-text:** Converts the hearing person's spoken response into text.
- **Avatar/fingerspelling:** Shows known word animations or fingerspells unknown words for visual feedback.

Why avatar is lightweight, not full ASL generation

A full ASL avatar would need grammar, facial expressions, non-manual markers, sentence structure, and detailed animation. That is beyond the current capstone hardening scope. VoxGest therefore uses a lightweight visual response prototype: known words can use simple animation files; unknown words can be fingerspelled. This is honest, manageable, and still connected to bidirectional communication.

Phrase Intents and Why They Are Frozen

Phrase intents were explored to map a complete gesture sequence to a sentence output such as ASK_NAME -> What is your name? This is useful, but it is structurally different from short word recognition.

Why short words and phrases must be separated

A short word can be recognized from a rolling 30-frame window. A phrase must be recognized only after the full phrase gesture is complete. If a phrase is classified too early, the system may output the sentence before the signer finishes the gesture. This happened in the first phrase-intent test and led to the endpoint segmentation design.

Correct phrase path

IDLE

-> START_MOTION

-> ACTIVE_MOTION

-> END_HOLD

-> resample complete segment to 60 frames

-> phrase model inference

-> output mapped sentence only if accepted

Phrase recognition is not deleted. It is frozen during Recognition Hardening v1. That means it is a future path, not the current default demo priority.

FullSign225 and Residual Dilated TCN Experiments

FullSign225 and Residual Dilated TCN show that the project has a future improvement path, but they must not be mixed with the protected baseline until tested. A good capstone system can have experiments, but it must label them honestly.

FullSign225

FullSign225 means pose 99 + left hand 63 + right hand 63 = 225 features per frame. The logic is that some signs may need both hands or full-body context. This can preserve information that onehand162 masks out. However, it increases input size, data requirements, and model complexity. Therefore, it is experimental and should stay separate from demo10 until live testing proves reliability.

Residual Dilated TCN

A Residual Dilated TCN is a stronger temporal convolution model. Dilation lets the model see farther across the 30-frame gesture without making the network huge. Residual connections help information survive through deeper

layers. This may help signs where the start, middle, and end all matter. But it cannot fix bad data, missing landmarks, or wrong hand metadata. It should be compared using validation reports and live logs before promotion.

Experiment	Why relevant	Why not default yet	Safe rule
FullSign225	May improve two-hand/full-body signs.	Needs clean multi-person data and live proof.	Keep separate profile and do not overwrite demo10.
Residual Dilated TCN	May learn wider motion patterns.	Needs comparison against basic TCN and live behavior.	Train/export as separate artifact.
Phrase TCN	Can map complete phrase gesture to sentence.	Premature recognition risk and data needs.	Keep disabled until endpoint data is stable.
ST-GCN/Transformer	Powerful research direction for skeleton/sequence modeling.	Too complex/data-heavy for current 2-week/chapter scope.	Mention as future work only.

Paper Holes and How to Fix/Explain Them

A paper hole is a place where the manuscript may invite criticism because it says something outdated, too broad, or disconnected from the actual system.

Potential hole	Why it is a problem	Defense-ready fix
Paper says CNN is the main final model.	The current system uses landmarks and temporal models.	Say CNN was an early approach and literature basis; current recognition uses landmark features.
Paper says only static alphabet.	Current system includes selected dynamic words and NOTHING.	Update scope/objectives/methodology to include static + dynamic + no-output class.
Paper claims full ASL translation.	System does not handle full grammar or unlimited signs.	Use selected sign recognition and controlled accessibility prototype wording.
Phrase intent presented as finished.	Phrase recognition is currently disabled during hardening.	Present it as explored/future pathway; token composer is current sentence strategy.
Avatar described like full sign avatar.	Current avatar is lightweight prototype.	Describe known-word animation and fingerspelling fallback only.
Android implied to train models.	Training remains in Python.	State Android consumes exported TFLite model, labels, and runtime manifest.
Validation accuracy used as only proof.	Live performance can differ.	Add live logger, negative testing, and Android readiness testing.
FullSign225 mixed with baseline.	Could confuse current defense scope.	Label FullSign225/RD-TCN as experimental and separate from protected demo10.

Defense-Ready Technical Answers

Why did you not stay with CNN?

Because CNNs learn from raw images and became sensitive to lighting, background, camera angle, hand distance, and cropping. VoxGest needs live camera reliability, so landmarks became the main feature strategy.

Why landmarks?

Landmarks turn the hand and body into coordinates. This makes the model focus on geometry instead of the whole room. It also makes Python and Android preprocessing more controllable.

Why 63 features?

Static signs mainly need hand shape. One hand has 21 landmarks and each has x, y, z coordinates, so $21 \times 3 = 63$.

Why 30 x 162?

Dynamic words need motion over time and body-relative location. 30 frames capture short motion. 99 pose features plus 63 hand features give the model body context and hand motion.

Why NOTHING?

Without NOTHING, the model may force idle or partial movement into a word. NOTHING teaches the system when no valid dynamic word should be displayed, spoken, or animated.

Why token composer?

The composer prevents raw unstable predictions from immediately becoming user-facing text. Only accepted tokens update the sentence.

Why Android/TFLite?

The goal is a portable, offline, on-device communication tool. TFLite lets Android run recognition locally without a backend.

Why not full ASL translation?

Full ASL grammar needs larger vocabulary, grammar modeling, facial expressions, and sentence-level understanding. VoxGest is currently a controlled assistive prototype.

Why phrase recognition disabled?

The first phrase test showed early output risk. Phrases need endpoint segmentation and more data. During hardening, stable words and token output are more important.

Why FullSign225 experimental?

It may improve signs needing both hands/body context, but it needs separate testing and must not overwrite the protected demo baseline.

Current Limitations and Next Engineering Path

Current limitations

- The system is not a complete ASL grammar translator.
- The vocabulary is controlled and limited.
- Some dynamic labels still require repair data and live testing.
- Android real-time inference must mirror Python preprocessing exactly.
- Avatar rendering is lightweight and not full linguistic sign animation.
- Phrase-level recognition exists as architecture but is disabled by default during hardening.
- FullSign225/RD-TCN are experimental and need separate validation.

Recommended next path

26. Keep demo10 protected as the safe baseline.
27. Finish hard-negative NOTHING samples and weak-label repair recordings.
28. Audit the dataset before each retraining cycle.
29. Train/compare TCN and LSTM using grouped validation and live logs.
30. Keep FullSign225/RD-TCN in separate experiment folders and never make them default without proof.
31. Integrate Android inference only after runtime contract matching is verified.
32. Collect ISO/IEC 25010 evaluation data for Chapter 4.
33. Use limitations honestly in Chapter 1 and Chapter 3.

Final Logical Summary

The logical creation of VoxGest follows a clear chain: the real-world problem is communication access; the platform choice is Android because it is portable; the recognition method is camera-based because it avoids gloves or special sensors; the feature method is landmarks because raw images were too fragile; the static model uses 63 features because hand shape is enough for alphabet/control signs; the dynamic model uses temporal landmarks

because moving signs need time; NOTHING and gates exist because raw model guesses are not safe enough; token composer exists because only accepted predictions should become text; speech and avatar exist because communication must be two-way; Android/TFLite exists because the system must run locally; and experiments such as FullSign225, RD-TCN, and phrase intents exist because the project has a future path but must not break the current defense-ready scope.

Final donkey-level conclusion

VoxGest did not change direction because the team was lost. It changed direction because each live test exposed a real weakness, and each new decision fixed a specific weakness. That is the correct engineering story: build, test, find the hole, fix the hole, protect the working baseline, then expand carefully.

Technical Pipeline Traces

This section traces what happens to one user action from camera input to final output. A senior reviewer can use this to check whether the architecture has unnecessary gaps.

Trace A: Static letter accepted into sentence

User signs letter H

- > Camera frame captures hand
- > MediaPipe detects 21 hand landmarks
- > Feature extractor subtracts wrist and scales by hand size
- > Static model receives [1, 63]
- > Model predicts H with confidence above threshold
- > Stable-frame logic confirms the prediction
- > Token composer appends H to current spelled text
- > UI shows updated sentence strip
- > Optional TTS speaks only when user triggers or system confirms output

Trace B: Dynamic word accepted into sentence

User signs WATER

- > Camera captures continuous frames
- > MediaPipe extracts pose + dominant hand landmarks per frame
- > Feature extractor anchors body/hand to pose reference and builds 162 features per frame
- > Rolling buffer reaches 30 frames
- > TCN/LSTM receives [1, 30, 162]
- > Model predicts WATER
- > Gates verify confidence, margin, motion, wrist path, hand presence, and stability
- > Token composer appends WATER as one full word token
- > UI updates; TTS/avatar can consume the confirmed token

Trace C: NOTHING blocks false output

User moves hand casually or pauses

- > Landmarks may still be visible
- > Dynamic model may output NOTHING or gates may reject weak motion

- > NOTHING is treated as no-op
- > Token composer does not change text
- > TTS does not speak
- > Avatar does not animate

Why this matters

The system has a safety chain. A model guess is not equal to a message. A message only exists after acceptance and composition.

Preprocessing: The Hidden Part That Can Break the Whole System

Preprocessing is the conversion from camera frames into model-ready numbers. It is easy to underestimate, but it is one of the most important parts of VoxGest. If preprocessing changes between training and Android runtime, the same sign may become a different number pattern.

Static preprocessing

- **Input source:** One detected hand.
- **Landmark count:** 21 hand landmarks.
- **Feature size:** 63 floats.
- **Normalization:** Subtract wrist landmark so hand position matters less.
- **Scaling:** Divide by a hand-size distance so near/far camera distance matters less.
- **Output:** A consistent [1, 63] tensor for A-Z/control recognition.

Dynamic preprocessing

- **Input source:** Pose landmarks and selected/dominant hand landmarks.
- **Frame feature size:** 99 pose floats plus 63 hand floats = 162.
- **Sequence length:** 30 frames for short dynamic words.
- **Spatial context:** Pose helps the system know if the hand is near face, chin, chest, or torso.
- **Single-hand policy:** Keep selected arm/body references and mask unrelated movement when enabled.
- **Output:** A [1, 30, 162] tensor for dynamic word recognition.

Common preprocessing failure modes

Failure mode	What happens	Prevention
Wrong mirror mapping	Left and right hands are interpreted backwards.	Map physical hand to MediaPipe label based on whether camera frame is mirrored.
Wrong landmark order	Model receives valid numbers but in the wrong meaning/order.	Freeze a runtime manifest and duplicate Python feature order in Android.
Missing scale normalization	The same sign changes when hand is closer/farther.	Use the same wrist/landmark scale rule.
Auto-hand during recording	Samples mix left/right behavior unexpectedly.	Use exact right or left hand in controlled recording.
Old metadata-less samples	Training mixes incompatible assumptions.	Archive/re-record or apply strict sample policy.

Training, Validation, and Live Testing Relationship

A technical reviewer may ask why the system still needs live testing if training reports already exist. The answer is that the model has to survive the real environment, not only saved arrays.

Test type	What it proves	What it does not prove
Training accuracy	The model can fit examples it sees during training.	It does not prove generalization.
Validation accuracy	The model can classify held-out saved	It does not fully prove live camera reliability.

	samples.	
Grouped validation	The model can generalize across recording groups better than random split.	Still may not reflect live Android camera behavior.
Diagnostic script	Shows confidence, margin, motion, hand presence, and acceptance behavior.	It is still a controlled test environment.
Live logger	Shows real-time accept/reject behavior and weak labels.	Needs repeated sessions and users for strong evidence.
ISO/IEC 25010 evaluation	Shows software quality feedback from evaluators.	It depends on proper testing tasks and respondent selection.

Why grouped validation matters

If samples from the same recording session appear in both training and validation, the model may only learn the session style. Grouped validation is better because it tests the model on separate sessions or groups. This matters for VoxGest because a sign can look different across time, hand speed, camera distance, and user fatigue.

Why live logs matter

Live logs reveal whether a failure came from low confidence, low margin, too little motion, weak wrist path, missing hand presence, or landmark instability. This helps the team repair the correct problem instead of randomly adding more data.

Relevance and Irrelevance of Alternative Approaches

Not every impressive AI method is the right method for the current capstone stage. This section explains what is relevant now, what is future work, and what is irrelevant or risky for the current defense-ready scope.

Approach	Status for current VoxGest	Reason
Raw CNN-only recognition	Historical / limited	Useful starting point and literature basis, but not robust enough as current live pipeline.
Landmark-based recognition	Core approach	Best balance of controllability, mobile suitability, and live robustness.
LSTM	Comparison/back-up	Good sequence model, but TCN is current main dynamic direction.
Basic TCN	Current main dynamic direction	Fits fixed-length sequences and mobile export better.
Residual Dilated TCN	Experimental	Promising stronger temporal model; requires separate proof.
FullSign225	Experimental	Preserves both hands and pose; may improve full signs but increases data/model demands.
Phrase intent TCN	Frozen future path	Needs endpoint segmentation and more data; not current default.
Cloud AI inference	Not aligned now	Conflicts with offline/privacy/latency goal.
Gloves or wearable sensors	Not aligned now	Adds hardware cost and reduces accessibility.
Full ASL grammar translation	Future research only	Needs grammar, non-manual markers, larger vocabulary, and stronger linguistic modeling.
Transformer / ST-GCN	Future research only	Could be powerful but too data-heavy and complex for the current hardening sprint.

Chapter 1 Detailed Alignment Notes

Chapter 1 should explain the problem, propose the solution, define scope, and prevent overclaiming. It should sound clear to non-technical readers but must not lie about the current system.

What Chapter 1 should emphasize

- Communication barriers exist in daily, healthcare, school, public service, and emergency-like situations.
- VoxGest is assistive, not a replacement for interpreters.
- The system is bidirectional: sign-to-text/speech and speech-to-text/avatar response.

- The recognition scope is controlled: static signs, selected dynamic words, NOTHING, token composer.
- The Android goal is offline on-device use through exported models.
- FullSign225, RD-TCN, and phrase intents are improvement paths, not current default claims.

Chapter 1 must avoid

- Avoid saying the system translates full ASL grammar.
- Avoid saying CNN is the current final model.
- Avoid saying dynamic recognition is only future if it is already part of the pipeline.
- Avoid saying avatar is complete ASL animation.
- Avoid saying Android trains the AI models.

Chapter 2 Detailed Alignment Notes

Chapter 2 should not be a random collection of AI studies. Each subsection must support a real design decision in VoxGest.

RRL topic	Why it belongs	Which system decision it supports
Assistive communication	Explains why communication support matters.	Bidirectional accessibility goal.
CNN / deep learning	Explains early image recognition and static classification background.	Historical model evolution and related work.
MediaPipe / landmarks	Supports converting camera frames to hand/body coordinates.	Core feature strategy.
Sequence models	Supports motion recognition over time.	LSTM/TCN dynamic words.
Negative class / rejection	Supports avoiding false outputs.	NOTHING and gates.
Edge computing / TFLite	Supports offline Android inference.	Android deployment.
Avatar / speech-to-text	Supports two-way communication.	Bidirectional output design.
ISO/IEC 25010	Supports evaluation categories.	Chapter 3 evaluation method.

Chapter 3 Detailed Alignment Notes

Chapter 3 must be the most technical chapter because it explains how the artifact is built. A panel can accept limitations if the methodology is clear, logical, and honest.

Required methodology chain

34. Research design: developmental research because the output is a working system.
35. Development methodology: Agile/RAD because the system evolved through repeated testing and revision.
36. Architecture: camera -> landmarks -> models -> gates -> composer -> speech/avatar.
37. Dataset development: static, dynamic, manual calibration, NOTHING, and experimental data paths.
38. Feature engineering: 63 static, 30 x 162 dynamic, 30 x 225 experimental FullSign225, 60 x 162 phrase path.
39. Model development: CNN, hybrid, landmark static, LSTM, TCN, RD-TCN experiment.
40. Recognition hardening: NOTHING, thresholds, stability, live logs.
41. Android development: Kotlin, CameraX, MediaPipe, TFLite, runtime manifest.
42. Testing: static, dynamic, negative class, live logs, Android readiness.
43. Evaluation: ISO/IEC 25010 with functional suitability, performance efficiency, reliability, usability.

Engineering Risk Register

Risk	Impact	Mitigation
Dataset mismatch	Model works in saved data but fails live.	Record webcam calibration samples and use live logger.
False positives	App speaks/displays wrong words.	Use NOTHING, gates, stability, cooldown, and token composer.
Android preprocessing mismatch	Correct model file behaves incorrectly on phone.	Use runtime manifest and mirror Python feature order exactly.
Vocabulary expansion too early	More labels increase confusion.	Freeze expansion until demo10 + NOTHING are stable.
Experiment overwrites baseline	Safe demo breaks.	Separate demo10, FullSign225, and RD-

		TCN artifacts.
Overclaiming in paper	Panel challenges credibility.	Use controlled prototype wording and honest limitations.
Weak avatar claim	Panel thinks avatar is full ASL.	Call it lightweight visual response/fingerspelling prototype.
Inconsistent team recordings	Training data becomes noisy.	Use recorder protocol, exact hand mode, metadata, and central merge/audit.

Glossary for Technical Review

Term	Simple explanation	Technical meaning in VoxGest
Landmark	A dot on the hand/body.	A coordinate point from MediaPipe.
Feature	A number the model reads.	A normalized coordinate value used as model input.
Tensor	A shaped box of numbers.	Model input array such as [1, 63] or [1, 30, 162].
CNN	Image pattern learner.	Early raw-image classifier, not current dynamic core.
LSTM	Step-by-step motion reader.	Sequence model used for dynamic comparison/back-up.
TCN	Motion pattern reader using time filters.	Current main dynamic training direction.
RD-TCN	Stronger TCN experiment.	Uses dilation and residual links for wider motion context.
NOTHING	The system should stay quiet.	No-output negative class.
Gate	Security guard.	Acceptance rule before user-facing output.
Token composer	Sentence builder.	Builds text from accepted labels only.
TFLite	Phone-ready AI file.	TensorFlow Lite model format for Android inference.

Source Basis Used

This document was rebuilt from the current VoxGest sources and prior paper updates, especially: ML Engineer Knowledge Base, Android Developer Handoff, README/Current Demo System, VoxGest Progress Logs, Sentence Intent Next-Level documentation, Avatar Pipeline, Chapter 2-3 Research Notes, and the updated Chapter 1-3 manuscript draft. It intentionally leaves the simplified contextual notes unchanged.